

# Factory Design Pattern Revision Notes

## Definition

Factory is a creational design pattern that encapsulates object creation logic. Instead of creating concrete classes directly inside business code, the client asks a factory for the required object.

Simple meaning: ask for what you need, let the factory decide how to create it.

## Why Factory matters

Factories are useful because object creation can become business-sensitive:

- Which payment provider should be used?
- Which notification sender should be used?
- Which exporter should be used?
- Which tenant-specific implementation should be loaded?
- Which credentials and configuration are needed?

If this logic is scattered across services, changing providers becomes risky. Factory centralizes creation.

## Factory types

### Simple Factory

A method or class creates an object based on input.

Use it when product selection is straightforward.

### Factory Method

A base class defines a creation method, and subclasses decide which concrete product to create.

Use it when subclasses represent different workflows but share an overall process.

### Abstract Factory

A factory creates families of related products.

Use it when products must be compatible as a set.

## When to use

Use Factory when:

- Object creation has branching logic.
- Constructors need configuration, credentials, or setup.
- The caller should not know concrete class names.
- Products are chosen by request type, tenant, provider, environment, or feature flag.
- Related objects must be created consistently.
- You want business services to focus on workflow, not construction.

Avoid Factory when:

- Object creation is a single obvious new.
- There is no variation or setup.
- A factory would only hide simple code.

## Where to use

Common business use cases:

- Payment gateway creation
- Notification sender creation
- Report exporter creation
- Database or cache client creation
- Tenant-specific services
- UI component families
- Parser or validator selection
- Cloud provider adapters

## JavaScript example: Simple Factory

```
class EmailSender {
  send(to, message) {
    console.log("Email to " + to + ": " + message);
  }
}

class SmsSender {
  send(to, message) {
    console.log("SMS to " + to + ": " + message);
  }
}

class NotificationFactory {
  static create(type) {
    if (type === "sms") {
      return new SmsSender();
    }

    return new EmailSender();
  }
}

const sender = NotificationFactory.create("sms");
sender.send("+8801000000000", "Order confirmed");
```

## Java example: Simple Factory

```
interface NotificationSender {
  void send(String to, String message);
}

class EmailSender implements NotificationSender {
  public void send(String to, String message) {
    System.out.println("Email to " + to + ": " + message);
  }
}

class SmsSender implements NotificationSender {
  public void send(String to, String message) {
    System.out.println("SMS to " + to + ": " + message);
  }
}
```

```

class NotificationFactory {
    static NotificationSender create(String type) {
        if ("sms".equals(type)) {
            return new SmsSender();
        }

        return new EmailSender();
    }
}

```

## JavaScript example: Factory Method

```

class ReportExporter {
    export(data) {
        const formatter = this.createFormatter();
        return formatter.format(data);
    }
}

class PdfReportExporter extends ReportExporter {
    createFormatter() {
        return new PdfFormatter();
    }
}

class CsvReportExporter extends ReportExporter {
    createFormatter() {
        return new CsvFormatter();
    }
}

```

## Java example: Factory Method

```

abstract class ReportExporter {
    public String export(List<Row> data) {
        Formatter formatter = createFormatter();
        return formatter.format(data);
    }

    protected abstract Formatter createFormatter();
}

class PdfReportExporter extends ReportExporter {
    protected Formatter createFormatter() {
        return new PdfFormatter();
    }
}

class CsvReportExporter extends ReportExporter {
    protected Formatter createFormatter() {
        return new CsvFormatter();
    }
}

```

## JavaScript example: Abstract Factory

```

class WebUiFactory {
    createButton() {
        return new WebButton();
    }

    createDialog() {
        return new WebDialog();
    }
}

class MobileUiFactory {
    createButton() {
        return new MobileButton();
    }

    createDialog() {

```

```

        return new MobileDialog();
    }
}

function renderCheckout(factory) {
    const button = factory.createButton();
    const dialog = factory.createDialog();
    button.render();
    dialog.open();
}

```

## Pros

- Centralizes object creation logic.
- Reduces coupling to concrete classes.
- Keeps business services focused on workflow.
- Makes provider selection easier to change.
- Improves testability by replacing products or factories.
- Helps maintain consistent families of related objects.

## Cons

- Adds extra files, methods, or classes.
- Can be over-engineered for simple construction.
- Simple factories can grow into large switch statements.
- Bad factory names can hide important behavior.
- Too much factory usage can become similar to a service locator.

## Business logic efficiency

Factory improves business logic by separating creation decisions from business workflows.

Example: Order confirmation workflow:

1. Mark order as confirmed. 2. Choose notification sender. 3. Send confirmation. 4. Save order status.

The workflow should not know SMTP keys, SMS tokens, push credentials, or vendor-specific constructors. A factory owns those details.

Business benefits:

- Faster provider changes.
- Lower regression risk.
- Cleaner service classes.
- Easier tenant-specific behavior.
- Easier testing with fake products.

## Interview question and answer

### Q1. What is the Factory design pattern?

Answer: Factory is a creational pattern that encapsulates object creation so clients do not directly depend on concrete construction details.

## Q2. What problem does Factory solve?

Answer: It prevents object creation logic from being scattered across the codebase. This makes provider changes, setup changes, and product selection easier to maintain.

## Q3. Simple Factory vs Factory Method?

Answer: Simple Factory uses one method to choose and create an object. Factory Method lets subclasses decide which product to create.

## Q4. Factory Method vs Abstract Factory?

Answer: Factory Method creates one product through subclass-specific creation. Abstract Factory creates families of related products through a common factory interface.

## Q5. Is Simple Factory a GoF design pattern?

Answer: Simple Factory is commonly used but is not one of the original GoF patterns. Factory Method and Abstract Factory are GoF patterns.

## Q6. Which SOLID principles does Factory support?

Answer: Factory can support Dependency Inversion by returning abstractions instead of concrete classes. It can support Open Closed Principle when new factories or creators are added without modifying business workflow.

## Scenario-based question and answer

### Scenario 1

Question: An order service creates EmailSender, SmsSender, and PushSender using if/else. What would you do?

Answer: Move creation into NotificationFactory. The order service asks the factory for a NotificationSender and then calls send.

### Scenario 2

Question: A report service creates PDF, CSV, and XLSX exporters. The business asks for XML export. Which factory style fits?

Answer: A Simple Factory fits if selection is based on requested format. Add XmlExporter and register it in the exporter factory. If each exporter has its own workflow subclass, Factory Method may be better.

### Scenario 3

Question: A UI app needs matching buttons, dialogs, and menus for web and mobile. Which factory pattern fits?

Answer: Abstract Factory, because it creates a family of compatible products for each platform.

### Scenario 4

Question: A service creates one User object with no configuration or branching. Should you add a factory?

Answer: Probably not. Direct construction is clearer when creation is simple and stable.

## Problem-solving question and answer

### Problem 1

Question: Add bKash payment gateway without changing CheckoutService.

Answer:

1. Create a PaymentGateway interface. 2. Implement BkashGateway. 3. Add bKash creation to PaymentGatewayFactory. 4. Make CheckoutService depend on PaymentGateway. 5. Select the gateway from request, configuration, or tenant settings.

## Problem 2

Question: Your factory has 20 if/else branches. How do you improve it?

Answer:

- Replace if/else with a registry map.
- Split factories by bounded context.
- Move large object graph wiring to dependency injection configuration.
- Keep each factory focused on one product family.

JavaScript registry example:

```
const creators = {  
  email: () => new EmailSender(),  
  sms: () => new SmsSender(),  
  push: () => new PushSender()  
};  
  
function createSender(type) {  
  const creator = creators[type] ?? creators.email;  
  return creator();  
}
```

## Problem 3

Question: How do you safely choose a product from user input?

Answer: Use a whitelist map of allowed keys. Do not dynamically evaluate class names from raw user input.

## Problem 4

Question: Two tenants need different email, payment, and invoice implementations that must match. Which pattern should you use?

Answer: Use Abstract Factory. Each tenant factory returns a compatible family such as EmailSender, PaymentGateway, and InvoiceFormatter.

## Fast memory hooks

- Factory hides creation.
- Client asks by intent.
- Product is the created object.
- Simple Factory selects from input.
- Factory Method delegates creation to subclasses.
- Abstract Factory creates compatible product families.
- Avoid factories for one simple stable constructor.